

Exploring Large Language Models for Verilog hardware design generation

Erik H. D'Hollander
Department of Electronics and
Information Systems
Ghent University
Ghent, Belgium
Erik.DHollander@UGent.be

Ewout Danneels
School of Computer Science
Engineering
Ghent University
Ghent, Belgium
Ewout.Danneels@UGent.be

Karel-Brecht Decorte
School of Computer Science
Engineering
Ghent University
Ghent, Belgium
KarelBrecht.Decorte@ugent.be

Senne Loobuyck
School of Computer Science
Engineering
Ghent University
Ghent, Belgium
Senn.Loobuyck@UGent.be

Arne Vanheule
School of Computer Science
Engineering
Ghent University
Ghent, Belgium
Arne.Vanheule@UGent.be

Ian Van Kets
School of Computer Science
Engineering
Ghent University
Ghent, Belgium
Ian.VanKets@UGent.be

Dirk Stroobandt
Department of Electronics and
Information Systems
Ghent University
Ghent, Belgium
Dirk.Stroobandt@UGent.be

Abstract—Deep Learning Large Language Models (LLMs) have the potential to automate and simplify code writing tasks. One of the emerging applications of LLMs is hardware design, where natural language interaction can be used to generate, annotate, and correct code in a Hardware Description Language (HDL), such as Verilog. This work provides an overview of the current state of using LLMs to generate Verilog code, highlighting their capabilities, accuracy, and techniques to improve the design quality. It also reviews the existing benchmarks to evaluate the correctness and quality of generated HDL code, enabling a fair comparison of different models and strategies.

Keywords—large language models, Verilog, AI, fine-tuning, prompt engineering, pass@k metric.

I. INTRODUCTION

Over the past decade, Deep Learning (DL) has impacted the world, e.g., through the use of Convolutional Neural Networks (CNNs) for image analysis and Recurrent Neural Networks (RNNs) for processing sequential data, such as speech recognition. The introduction of transformers in 2017 by Vaswani et al. [1] marked a new significant shift in natural language processing. Transformers use a self-attention mechanism to assess the importance of different parts of a sequence, allowing for the detection of word dependencies over long distances without relying on sequential processing like RNNs. This was further advanced by Google's BERT in 2018, which introduced bidirectional training to better understand the context surrounding each word by analyzing text from both directions, unlike the original transformer model that only considered the preceding context. In the same vein, OpenAI released GPT-1 in 2018, initiating the series of Generative Pre-trained Transformers (GPT) that utilize generative pre-training to predict the next word in a sentence, significantly enhancing the model's capability to understand long-range dependencies. This foundation was built upon by subsequent versions, including GPT-2, GPT-3, GPT-3.5, and GPT-4, which improved performance on various natural language processing tasks by expanding the model size and training datasets. These advancements have facilitated the use of Large Language Models (LLMs) in tasks such as code generation, allowing for the creation of code in multiple programming languages from natural language descriptions. When applied to Hardware Description Languages (HDLs) such as Verilog, LLMs could significantly speed up the generation of hardware designs used in reconfigurable

architectures like FPGAs. Compared to traditional languages such as Python, the task becomes more challenging due to the need for low level precision and a deep understanding of hardware architecture. This survey compares two methodologies for generating Verilog code with LLMs: fine-tuning and prompt engineering, examining their efficacy in identifying and fixing bugs and security vulnerabilities. It also discusses various benchmarks for assessing code generation techniques and briefly touches upon the environmental implications of training LLMs.

II. VERILOG CODE GENERATION USING LLMs

Verilog and other Hardware Description Languages are designed for defining the structure and behavior of digital circuits and systems. Unlike general-purpose programming languages such as Python, HDLs like Verilog operate at a lower level of abstraction, encompassing detailed hardware-specific features including concurrency, timing, signals, ports, and modules. Generating Verilog code with Large Language Models poses more difficulty than generating Python code due to Verilog's complex, hardware-oriented features. LLMs, trained on vast text corpora, learn the syntax, semantics, and style of languages but might lack sufficient data or understanding to produce Verilog code that is syntactically correct, semantically meaningful, and functionally valid. Capturing the intricacies of hardware design, such as timing constraints, optimization goals, and verification techniques, can be particularly challenging for LLMs. This section explores two approaches to improve Verilog code generation: fine-tuning the model and prompt engineering.

A. Fine-tuning LLMs

Enhancing code generation quality with a Large Language Model (LLM) can be achieved by modifying its pre-trained weights, a process known as fine-tuning. In an early attempt, Pearce et al. [2] fine-tuned the weights of DAVE, an LLM based on ChatGPT-2, using a dataset specifically tailored for Verilog. DAVE's primary challenge was its inherited output limitation from ChatGPT-2, capped at 1024 tokens. This restriction influenced design decisions, such as limiting sequence generators to four elements and avoiding detailed register templates to prevent exceeding the token limit. Testing DAVE and ChatGPT-3.5 with the same prompt showed both models could generate correct Verilog code for most of the validation templates. However, slightly altering the prompt could lead DAVE to produce incorrect Verilog

code [3], whereas ChatGPT-3.5, even without fine-tuning, generated accurate code and provided additional insights. E.g. in Fig. 1, the responses of DAVE and ChatGPT-3.5 to the prompt *For 'a' and 'b', check if these are both true and return the result in 'c'. when true is changed into 1, are compared.*

<i>For 'a' and 'b', check if these are both 1 and return the result in 'c'. gives the following answers:</i>	
DAVE:	<code>assign c = 1 & b;</code>
ChatGPT3.5:	<code>assign c = (a & b);</code>

Fig. 1 Comparing two LLMs with a slightly altered prompt

These findings indicate that recent and larger LLMs, like the newer ChatGPT versions, have enhanced the generation of Verilog code for a wider variety of prompts. Although DAVE is no longer at the forefront of technology, the research from 2020 underscored the potential of using LLMs for Verilog code generation, marking a significant step in the field.

B. Prompt engineering

Prompt engineering refers to the process of carefully designing and refining the input prompts given to an LLM, to generate the most useful, accurate, or creative responses. This skill is becoming increasingly important for tasks driven by artificial intelligence in the field of natural language processing (NLP). The following methods all implement prompt engineering to improve the code generation capabilities of an LLM.

1) Reinforcement Learning with Human Feedback

Instead of creating a fine-tuned model for Verilog code generation, Blocklove et al. [4] introduced a framework of conversational steps outlined in the Reinforcement Learning with Human Feedback (RLHF) scheme. The process begins by providing the LLM with a design prompt, including specifications. The LLM is then asked to propose a design that meets these specifications, and to create a testbench for this design. If the outcome fails, the prompt asks for code corrections from the LLM. If this fails, further prompts incorporate increasing amounts of human feedback for persistent errors. Blocklove et al. applied this method to several models including Bard, Huggingchat, ChatGPT-3.5, and ChatGPT-4, observing varied results in the task of generating a simple 8-bit shift register. Huggingchat produced nonsensical outcomes, while Bard encountered a correctable error. The GPT models, however, successfully generated functional shift registers, with ChatGPT-4 also providing comprehensible and useful comments. The team further challenged the LLMs to generate the complete code for an 8-bit microprocessor, a task only ChatGPT-4 managed to accomplish. Nonetheless, the need for extensive human intervention to rectify errors limits the efficiency of the design workflow.

2) Chain of thought prompting

The chain of thought (CoT) prompting is a technique for tackling complex prompts, by providing the model with a series of predefined questions and answers as a guide for solving similar problems. This approach is exemplified in research by Du et al., [5] where it is applied to generate Verilog code for a Fast Fourier Transformation (FFT), specifically to generate the twiddle factors that progress in a power-of-2 sequence. ChatGPT-3.5 was given instructions to compute twiddle factors up to the third power of 2, then asked to extend this to the fourth and fifth powers. The model accurately

generated these factors, as opposed to its failure when attempting the task without guidance. This underscores the effectiveness of CoT for producing Verilog code for iterative operations, provided the user has a prior knowledge of the subject.

3) Self-planning

Self-planning, introduced by Lu et al. [6], is a novel prompt engineering method that eliminates the need for human input or prior knowledge in the design process, involving two main phases: planning and coding. Initially, the LLM is given the natural language description of the design, as well as a template of the module name and input output signals. Using this information the LLM is asked to devise a coding strategy for the desired design, outlining the logic steps and natural language explanations, while also pinpointing potential syntax errors to avoid in Verilog code generation. Following this, the planning strategy formulated by the model is integrated with the initial design description as the enhanced LLM prompt to generate the Verilog design. According to a suite of 30 benchmarks by Lu et al., this approach improves the overall accuracy of syntax in code generated by ChatGPT-3.5+Self-Planning from 55% to 73% and functional correctness from 33% to 47%, nearly matching the results of base ChatGPT-4, which achieves 81% syntax accuracy and 50% functional correctness without fine-tuning or self-planning. Applying self-planning to ChatGPT-4 increases the accuracy to 90% and functional correctness to 63%.

4) Copilot

GitHub's Copilot, a code assistance tool trained on open-source code from GitHub repositories, offers code suggestions directly in code editors. Pearce et al. [7] conducted a study focusing on its application in Verilog code generation to identify its constraints. Designed with software coding in mind, Copilot faces challenges with hardware-specific Verilog, mainly due to insufficient training on hardware description languages. The study highlighted Copilot's attempt to use SystemVerilog, which merges features of C and Verilog, but noted its struggle with essential Verilog concepts like distinguishing between registers and wires. Pearce et al. concluded that Copilot's current training data, lacking in Verilog content, renders it less effective for generating Verilog code.

III. VERILOG CODE CORRECTION WITH LLMs

Large Language Models can be used to identify and fix errors in Verilog code, regardless of whether the code is manually written or produced by an LLM. This section discusses two systems to facilitate code correction: CWEAT and FLAG. These frameworks represent advancements in utilizing LLMs for the purpose of code repair.

A. CWEAT

In Fixing Hardware Security Bugs with Large Language Models, B. Ahmad et al. [8], utilize Large Language Models for repairing security vulnerabilities in hardware code. This approach involves generating a prompt that outlines the specific bug and solicits a corrective action from the LLM, subsequently assessing the effectiveness of the proposed solution. The bug detection model sources its data from ten Common Weakness Enumerations (CWEs) [9] associated with hardware security flaws, derived from three different collections. Four of these CWEs are from MITRE, including issues like Locked Register and Trustzone Peripheral, with simple design examples created to illustrate these weaknesses.

Three additional CWEs are sourced from Google's OpenTitan project, which focuses on implementing security features within the System on Chip to enhance its security. The researchers introduced deliberate bugs into these modules to simulate real-world vulnerabilities. The remaining three CWEs are drawn from the Hack@DAC-21 competition, highlighting bugs discovered in the hackathon's challenge focused on Real-Time Logic level vulnerabilities. The detection component employs a static analysis tool capable of identifying certain weaknesses within the RTL by analyzing Abstract Syntax Trees (ASTs) produced by the Verific Verilog parser. This tool searches for specific keywords and patterns indicative of vulnerabilities in these particular CWEs. The repair generator then uses information about the CWE and the bug's location from the sources and detector to craft a detailed prompt for the LLM. This prompt includes the original code, the problematic code highlighted in comments, and precise repair instructions, grouped into "Bug instruction" and "Fix instruction" sections to guide the repair process. Finally, the evaluator tests the LLM's output for both security and functionality using the same analysis tool as the detector. It achieves this by replacing the erroneous code with the corrected version and re-running the analysis. If the same issue is detected, the proposed fix is deemed incorrect. An example of LLM repair as presented in [8] is given in Fig. 2.

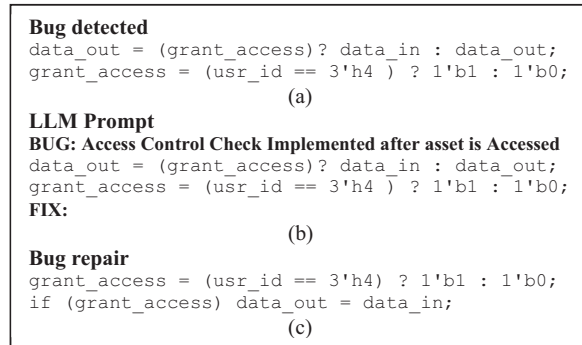


Fig. 2. Bug fixing by an LLM. (a) The bug is located using a CWE-tool, (b) a prompt is generated with the bug type, buggy code and fix instruction; (c) the repair code is generated.

B. FLAG

The FLAG (Finding Line Anomalies using Generative AI) system [10] uses an LLM for bug detection. It processes a Verilog program by extracting and individually regenerating each code line, then comparing the regenerated line with the original to identify discrepancies. The FLAG framework comprises several components that collectively assess whether a line contains errors, including Prompt Formation, Line Generation, Feature Extraction, and Classification. The Prompt Formation creates a prompt aimed at regenerating a specific line from the source code. Therefore, the source code file is divided into two sections surrounding the target line (prefix and suffix), each limited to 50 lines and retaining comments. The prompt combines these two sections. In the Line Generation phase, the formed prompt is submitted to an LLM. The Feature Extraction component compares the LLM-generated line with the original to identify features that aid in determining the line's accuracy. Key features include the Levenshtein distance used to calculate the similarity between the original and generated lines (ld) and their distance from comments (dfc). The Classification phase decides if a line is likely to be erroneous. A simple threshold-based rule

evaluates the Levenshtein distance: a slight increase suggests potential errors, whereas a zero distance indicates accuracy, and a significantly higher distance implies a different solution generated by the LLM, likely not an error. The distance from comment metric adjusts the classification severity based on the proximity of the line to the nearest comment, with closer lines undergoing stricter evaluation. Together, these metrics offer a systematic approach to anomaly detection. Using several LLMs and different criteria, the results of [10] show that on the average 77% of the true defects in Verilog are detected (True Positive Rate), whereas 28% of the code lines flagged as defective turn out to be false positives (False Positive Rate).

While the area of LLM-aided hardware code correction is relatively young, the two frameworks CWEAT and FLAG demonstrate the feasibility to correct basic errors in Verilog.

IV. EVALUATING LLMs FOR VERILOG CODE GENERATION

This section explores the pass@k metric as well as a number of benchmark systems for evaluating the performance of LLM generated Verilog code.

A. Pass@k metric

The pass@k metric, introduced by Chen et al. [11], assesses the accuracy of code generation. This metric involves generating k solutions for a problem and considering a problem as solved if any one of these samples passes the relevant unit tests. The pass@k score of a data set is the ratio of solved problems to the total number of problems in the set. A value k=1 calculates the proportion of problems solved on the first attempt, whereas a k=10 evaluates the success rate across 10 attempts. This allows the model more leniency to follow different reasoning steps. An extra advantage of the pass@k metric is its ability to accept a broad range of functionally equivalent solutions as correct. This means that a piece of generated code will be deemed correct if it is functionally equivalent to the "standard" solution, regardless of differences, as long as it successfully passes a series of unit tests to verify its functional equivalence.

B. Supervised Fine-Tuning

VerilogEval, introduced by Liu et al. [12], is a framework designed for assessing the capabilities of Large Language Models in handling Verilog code. It is structured around three key components: an evaluation dataset, an automated testing setup, and a dataset for Supervised Fine-Tuning (SFT) aimed at boosting the performance of CodeGen LLMs across various model sizes. The evaluation dataset features 156 problems sourced from the website HDLBits, focusing on self-contained modules, which Liu et al. describe as modules not requiring instantiation of other modules. Problems are represented as vectors, representing a problem description including a module header and IO definition, a question prompt, and an ideal, golden solution. Problem descriptions were generated through either manual efforts or LLM assistance, leading to the creation of two distinct datasets: VerilogEval-human and VerilogEval-machine. VerilogEval-machine refers to creating a synthetic dataset for supervised fine-tuning. In this approach, Large Language Models are fine-tuned on synthetic data generated by the models themselves. The synthetic part is to create a natural language problem description from the problem-code pairs of the original dataset, by utilizing the LLM to generate the problem description. This synthetic dataset is then used to fine-tune the pretrained language models, enhancing their capability to

generate Verilog code. SFT allows for the adaptation of LLMs to specific tasks or domains, such as Verilog code generation, by providing tailored training data that aligns with the requirements of the target task. The automated testing environment assesses code correctness by comparing simulation outcomes of generated code against golden solutions. To evaluate the impact of Supervised Fine-Tuning, experiments were conducted using various models, including the CodeGen series and GPT-3.5-turbo and GPT-4.0, on both VerilogEval-human and VerilogEval-machine datasets. SFT enhances the CodeGen-16B performance to a level close to the GPT models. In addition, results are significantly better with the VerilogEval-machine dataset. Analysis revealed a performance variance with the VerilogEval-human dataset, ranging from slight improvements to decreases, likely influenced by the nature of the SFT data generation. Observations also indicated potential model overfitting, as demonstrated by differing improvement rates across training epochs, and suggested that larger models tend to outperform smaller ones.

C. RTLLM

The RTLLM (RTL generation with Large Language Model) study by Lu et al. [6] introduces a new open-source benchmark specifically for generating Hardware Description Language code. This benchmark focuses on three objectives: syntax, functionality, and quality of the generated HDL code and contains 30 diverse hardware designs, varying in scale and complexity. Each design is accompanied by a natural language description to guide the model, a testbench comprising several test cases to verify the design's functional correctness, and a human-crafted reference design for quality comparison. The benchmarking procedure begins with a comprehensive natural language description detailing the design's full functionality. This description can be further refined with prompt engineering techniques such as self-planning. The code generated by an LLM is then assessed across the benchmark's three objectives. The evaluation starts with a syntax check, followed by a functionality score determined through testbench performance. Lastly, the generated design's quality is evaluated by comparing its performance, power, and area (PPA) metrics against those of the reference design. The best outcomes for average syntax correctness as well as functional correctness of two LLMs, with and without Self-Planning are shown in Table I, excerpt from [6].

TABLE I. BENCHMARK OF LLMs

Correctness	GPT-3.5	GPT-3.5 + SP	GPT-4	GPT-4 + SP
Syntax	55%	73%	81%	90%
Functionality	33%	47%	50%	63%

Although the parameters of many large models are not available for open access, preventing their fine-tuning, the application of self-planning prompt engineering almost doubled the quality of the generated Verilog code. Despite this, the low functional correctness makes it essential to adopt a cautious approach and thoroughly verify the resulting code. Furthermore, it is important to note that these advancements have been achieved quite recently as shown in the timeline of Fig. 3, and significant further improvements can be anticipated.

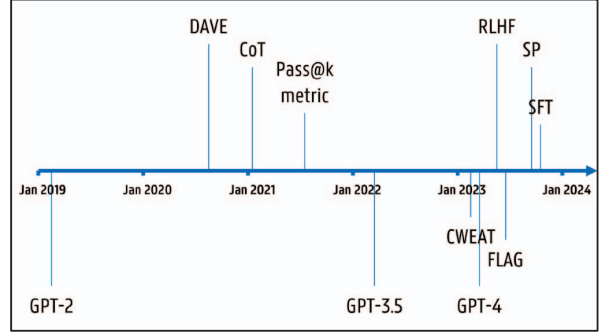


Fig. 3. Enhancing LLMs for Verilog: timeline.

V. CARBON FOOTPRINT

A study conducted by researchers at the University of Massachusetts, Amherst [13], estimated that training a single large language model can emit as much of the equivalent of nearly five times the lifetime emissions of an average car. The specific carbon footprint varies based on the energy consumption and the carbon intensity of the energy sources utilized. Given these environmental concerns, developers are advised to consider fine-tuning pre-existing models instead of building new ones from the ground up. This approach is environmentally friendlier as it demands fewer computational resources during the training phase.

VI. CONCLUSION

AI-assisted Hardware Description Language design is a hot research topic. Fine-tuning and prompt engineering have been identified as key techniques for enhancing the capabilities of Large Language Models for generating Verilog designs. Despite these improvements, there is an ongoing debate about the performance of open-source LLMs versus their closed-source counterparts, with the latter often outperforming the fine-tuned versions of open-source models. However, training general LLMs, such as GPT-4, presents substantial challenges, including the extensive time required and the considerable carbon footprint associated with their development. In addition, they generate only a marginal improvement over present alternatives. It is expected that further research on domain-specific LLMs may offer a more sustainable and effective route to advancing AI-assisted HDL design, while maintaining or improving the syntactical and functional accuracy.

REFERENCES

- [1] A. Vaswani et al., "Attention Is All You Need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, Long Beach California USA: ACM, Dec. 2017, pp. 6000–6010. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [2] H. Pearce, B. Tan, and R. Karri, "DAVE: Deriving Automatically Verilog from English," in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, Nov. 2020, pp. 27–32. doi: 10.1145/3380446.3430634.
- [3] N. Bild, "Dave, I Really Think I'm Entitled to an Answer to That Question (in Verilog)," Hackster.io. [Online]. Available: <https://www.hackster.io/news/dave-i-really-think-i-m-entitled-to-an-answer-to-that-question-in-verilog-9aa5305b42b7>
- [4] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and Opportunities in Conversational Hardware Design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, Snowbird, UT, USA: IEEE, Sep. 2023, pp. 1–6. doi: 10.1109/MLCAD58807.2023.10299874.

- [5] Y. Du, S. C. Liew, K. Chen, and Y. Shao, "The Power of Large Language Models for Wireless Communication System Development: A Case Study on FPGA Platforms." arXiv, Jul. 14, 2023. [Online]. Available: <http://arxiv.org/abs/2307.07319>
- [6] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTL2LLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model." arXiv, Sep. 26, 2023. doi: 10.48550/arXiv.2308.05345.
- [7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions." arXiv, Dec. 16, 2021. [Online]. Available: <http://arxiv.org/abs/2108.09293>
- [8] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "Fixing Hardware Security Bugs with Large Language Models." arXiv, Feb. 02, 2023. [Online]. Available: <http://arxiv.org/abs/2302.01215>
- [9] B. Ahmad *et al.*, "Don't CWEAT It: Toward CWE Analysis Techniques in Early Stages of Hardware Design," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, San Diego California: ACM, Oct. 2022, pp. 1–9. doi: 10.1145/3508352.3549369.
- [10] B. Ahmad, B. Tan, R. Karri, and H. Pearce, "FLAG: Finding Line Anomalies (in code) with Generative AI." arXiv, Jun. 21, 2023. [Online]. Available: <http://arxiv.org/abs/2306.12643>
- [11] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code." arXiv, Jul. 14, 2021. doi: 10.48550/arXiv.2107.03374.
- [12] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, San Francisco, CA, USA: IEEE, Oct. 2023, pp. 1–8. doi: 10.1109/ICCAD57390.2023.10323812.
- [13] E. Strubell, A. Ganesh, and A. McCallum, "Energy and Policy Considerations for Deep Learning in NLP." arXiv, Jun. 05, 2019. doi: 10.48550/arXiv.1906.02243.